SE 216 – SOFTWARE PROJECT MANAGEMENT
SOFTWARE TOOLS DOCUMENT

PROJECT NAME: NeuroMatter: Universal Local AI Hub

GROUP MEMBERS: Duru Deniz Alptekin, Şevval Acar, Kaan Efe Koygun, Ege Haktan Aykut, Ali Utku Aydın, Eren Karlı, Deren Kocabaş, Efe Berkay Soyucak.

TASK #	PROJECT TASKS WHICH REQUIRE SOFTWARE TOOL SUPPORT
1	User Interface Layer (Mobile Application)
2	Database Management System
3	AI Inference & Intelligence Management
4	Control Layer
5	Monitoring Layer

SOFTWARETOOLSFORTASK1: User Interface Layer

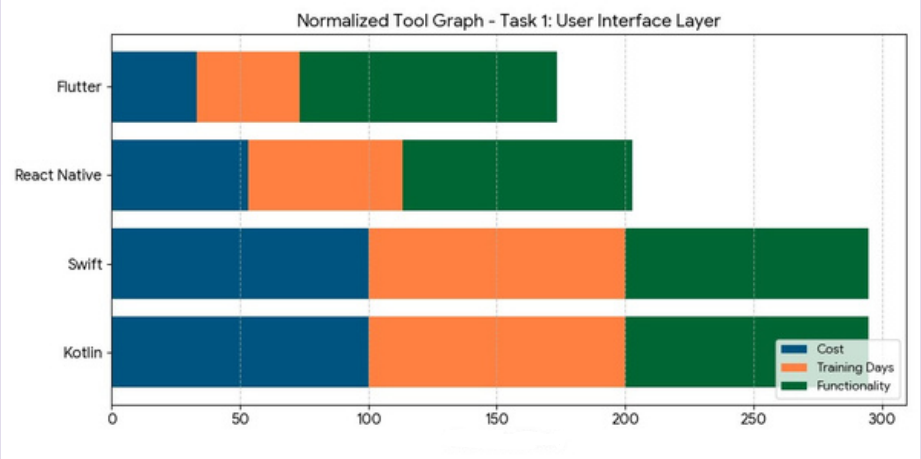
Tool Cost/Training/Functionality Data

Tool	*Flutter*	*React Native*	*Swift*	*Kotlin*
Cost	$500/y	$800/y	$1500/y	$1500/y
Training Days	10	15	25	25
Functionality	95	85	90	90

Normalized Cost/Training/Functionality Data

Tool	*Flutter*	*React Native*	*Swift*	*Kotlin*
Cost	33.3	53.3	***100***	100
Training Days	40	60	***100***	100
Functionality	100	89.4	***94.7***	94.7

Normalized Tool Graph



Which tool has been selected? Why?
When evaluating our options among Swift, Kotlin, and React Native, we have decided to move forward with Flutter. Our choice is rooted in a clear strategic advantage: Flutter provides the most practical and fastest solution by utilizing a single codebase for both iOS and Android. This approach allows us to deliver a high-quality experience to all users simultaneously, ensuring that our features remain consistent across different devices without the need for separate development teams or redundant effort.

By choosing Flutter over platform-specific languages or bridge-based frameworks, we significantly accelerate our development cycle and reduce operational overhead. Its high-performance rendering engine and "Hot Reload" capabilities allow us to iterate quickly and maintain a pixel-perfect UI on every screen. Ultimately, Flutter represents the most efficient path to market, offering a streamlined development process that combines the speed of cross-platform tools with the polished feel of a native application.

SOFTWARE TOOLS FOR TASK 2: Database Management System

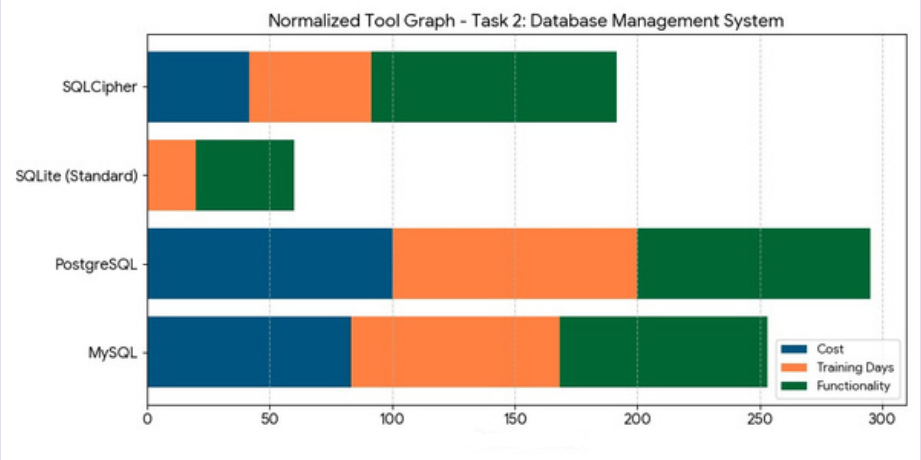
Tool Cost/Training/Functionality Data

Tool	*SQLCipher*	*SQLite (Standard)*	*PostgreSQL*	*MySQL*
Cost	$500/y	$0	$1200/y	$1000/y
Training Days	5	2	10	10
Functionality	100	40	90	85

Normalized Cost/Training/Functionality Data

Tool	*SQLCipher*	*SQLite (Standard)*	*PostgreSQL*	*MySQL*
Cost	41.6	0	***100***	83.3
Training Days	50	20	***100***	85
Functionality	100	40	***95***	85

Normalized Tool Graph



Which tool has been selected? Why?

For the data storage layer of our system, SQLCipher has been strategically selected to handle our local database needs. Given the "closed-box" nature and the strictly local operational principle of this project, ensuring that all data remains secure directly on the device is of critical importance. While standard SQLite offers advantages such as a minimal learning curve and ease of implementation, it lacks the native security features required to protect sensitive information from unauthorized access or external extraction.

To address these security requirements, SQLCipher provides a robust extension to SQLite by implementing 256-bit AES encryption. This level of industrial-grade encryption is the primary reason for our choice, as it ensures that local event logs and system data are fully encrypted at rest. By integrating SQLCipher, we maintain the lightweight efficiency of a relational database while providing a high-security environment that aligns perfectly with our project's commitment to data integrity and localized privacy.

SOFTWARE TOOLSFORTASK3: AI Inference & Intelligence Management

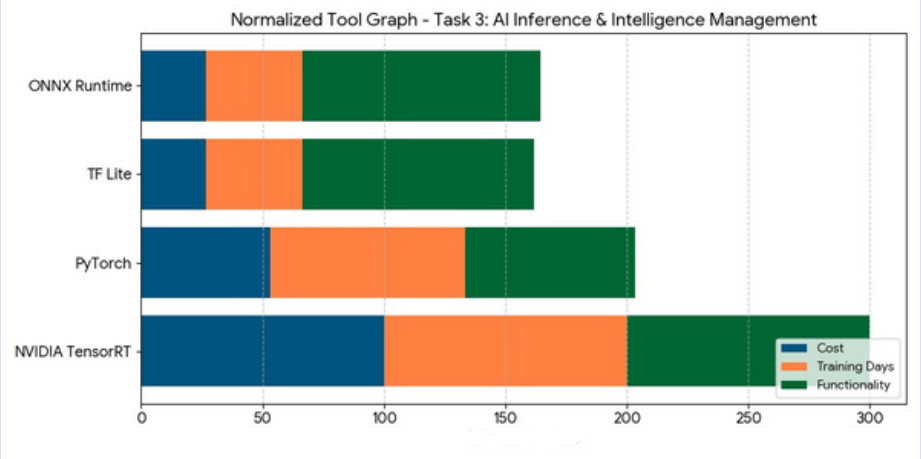
Tool Cost/Training/Functionality Data

Tool	*ONNX Runtime*	*TF Lite*	*PyTorch*	*NVIDIA TensorRT*
Cost	$400/y	$400/y	$800/y	$1500/y
Training Days	6	6	12	15
Functionality	98	95	70	100

Normalized Cost/Training/Functionality Data

Tool	*ONNX Runtime*	*TF Lite*	*PyTorch*	*NVIDIA TensorRT*
Cost	26.6	26.6	***53.3***	100
Training Days	40	40	***80***	100
Functionality	98	95	***70***	100

Normalized Tool Graph



Which tool has been selected? Why?
For the vision and intelligence layer of our system, we have strategically selected YOLOv8n for real-time object detection and Qwen2-VL-2B as our Vision Language Model (VLM). To ensure these models operate efficiently within our local infrastructure, they will be deployed using ONNX Runtime or TensorFlow Lite. This lightweight execution framework is an absolute requirement to meet our strict "2-second rule," ensuring that the system processes visual data and generates intelligent insights at maximum speed without relying on external cloud processing.

By leveraging these optimized formats, we can significantly reduce the computational footprint and memory usage of the models while maintaining high accuracy. YOLOv8n provides the lightning-fast detection needed for immediate environmental awareness, while Qwen2-VL-2B allows the system to understand complex visual contexts through natural language. Utilizing TFLite or ONNX quantization techniques allows us to squeeze every bit of performance out of the local hardware, guaranteeing that the end-to-end inference latency stays well below the 2-second threshold required for a seamless and responsive user experience.

SOFTWARE TOOLS FOR TASK 4: Control Layer

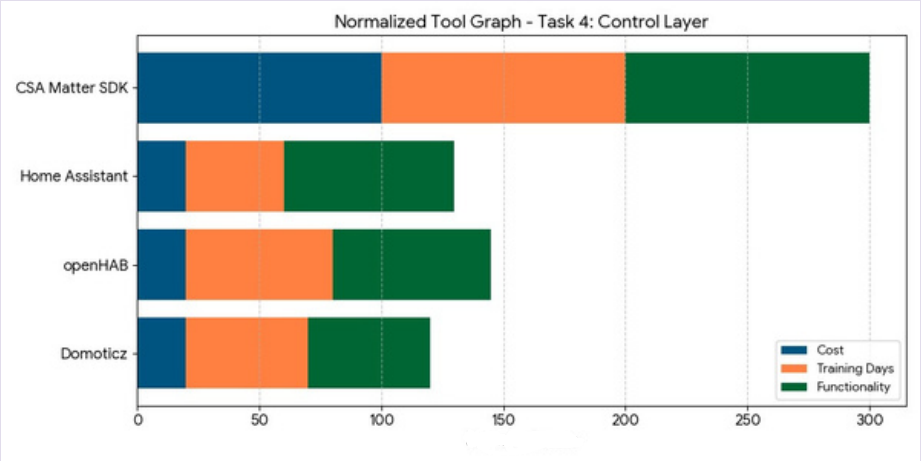
Tool Cost/Training/Functionality Data

Tool	*CSA Matter SDK*	*Home Assistant*	*openHAB*	*Domoticz*
Cost	$1000/y	$200/y	$200/y	$200/y
Training Days	20	8	12	10
Functionality	100	70	65	50

Normalized Cost/Training/Functionality Data

Tool	*CSA Matter SDK*	*Home Assistant*	*openHAB*	*Domoticz*
Cost	100	20	***20***	20
Training Days	100	40	***60***	50
Functionality	100	70	***65***	50

Normalized ToolGraph



Which tool has been selected? Why?

For the automation engine, we have strategically selected the CSA Matter SDK paired with a Python (FastAPI) backend. This combination allows us to establish a high-performance architecture where the core hardware layer communicates via C++ for near-zero latency, while Python serves as the sophisticated bridge between our artificial intelligence models and the physical hardware. By offloading time-critical tasks to C++ and leveraging FastAPI's asynchronous capabilities for high-level logic, the system achieves a professional-grade balance between raw power and modern flexibility.

Integrating the Matter protocol is a foundational decision for the project's long-term scalability and interoperability. As the industry standard for smart homes, Matter ensures that our system is not a closed ecosystem but a future-proof platform capable of seamless communication with a vast array of third-party smart devices. This standardized approach, combined with our custom automation engine, elevates the backend architecture to a level of performance and compatibility that meets the highest modern standards for smart infrastructure.

SOFTWARE TOOLSFORTASK5: Monitoring Layer

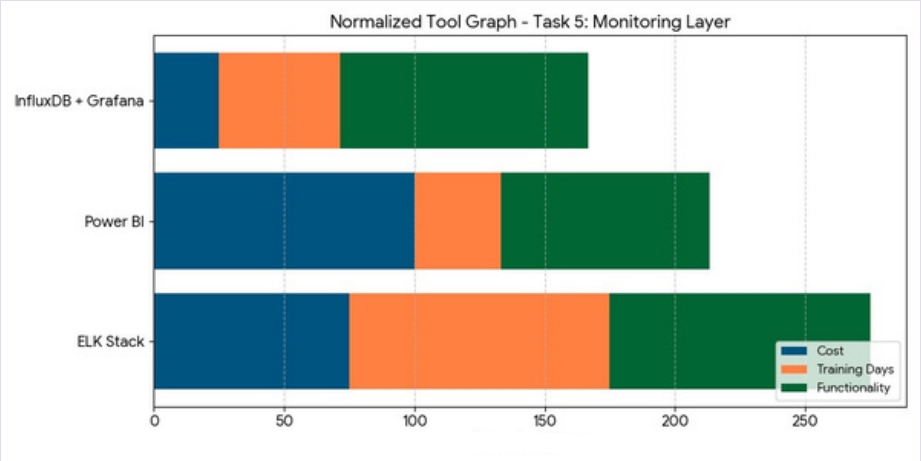
Tool Cost/Training/Functionality Data

Tool	*InfluxDB + Grafana*	*Power BI*	*ELK Stack*	
Cost	$300/y	$1200/y	$900/y	
Training Days	7	5	15	
Functionality	95	80	100	

Normalized Cost/Training/Functionality Data

Tool	*InfluxDB + Grafana*	*Power BI*	*ELK Stack*	
Cost	25	100	***75***	
Training Days	46.6	33.3	***100***	
Functionality	95	80	***100***	

Normalized Tool Graph



Which tool has been selected? Why?

The monitoring and analysis layer of our system is built upon a strategic architecture featuring InfluxDB, SQLCipher, and Grafana, specifically engineered to provide high-performance data processing within a local, "black box" environment. While InfluxDB serves as the core engine for handling high-velocity time-series sensor data with unmatched efficiency, SQLCipher is integrated to provide a secure and encrypted repository for event logs. This combination ensures that sensitive system logs and operational metadata are protected via 256-bit AES encryption, maintaining data integrity and confidentiality without needing an external security layer.

By intentionally avoiding heavy, cloud-dependent business intelligence tools such as Power BI, we have adopted a "clean data engineering" philosophy that prioritizes system autonomy and low latency. This approach allows for the seamless visualization of complex datasets directly at the edge through Grafana, ensuring that the system remains fully functional and highly responsive even in isolated or bandwidth-constrained settings. Consequently, this localized pipeline eliminates external dependencies, providing a streamlined, secure, and rapid visualization experience that is perfectly suited for mission-critical mission systems.